

Spring Core

Spring Core Module

Spring Core is the fundamental module of the Spring Framework. It provides the foundation for dependency injection and bean management.

- **IOC (Inversion of Control):** Instead of the developer controlling the creation and management of objects, the Spring container takes control.
- **DI (Dependency Injection):** Objects are given their dependencies rather than creating them inside the class. This reduces tight coupling.

Benefits

- Promotes loose coupling between classes.
- Increases reusability and maintainability.
- Makes unit testing easier by allowing dependencies to be mocked or swapped.

Spring Beans

- Java objects that are created, managed, and destroyed by the Spring IOC Container.
- Defined in configuration (XML, annotations, or Java config).

Why do we need Spring Core?

Normally in Java:

- Objects are created manually using new.
- Classes are **tightly coupled** → changing one requires changing others.
- Hard to maintain and test.

With Spring Core:

- Spring creates & manages objects.
- Classes become **loosely coupled** → easy to switch implementations.
- Improves reusability, flexibility, and testability.

Example for tightly coupled:

```
class Airtel {
    public int insert() {
        return 1; // logic
    }
}

class Sim {
    Airtel a = new Airtel(); // tightly coupled with Airtel

    public void calling() {
        int status = a.insert();
        if (status >= 1) {
            System.out.println("YOU CAN CALL..");
        } else {
            System.out.println("Trouble Calling..");
        }
    }
}

public class App {
    public static void main(String[] args) {
        Sim s = new Sim();
        s.calling();
    }
}
```

Problem:

- Sim always depends on **Airtel**.
- If you want to switch to **Jio**, you must change the Sim class code.
- This is **tightly coupled**.

Example for Loose coupling using spring:

```
interface Sim {
    void calling();
}

class Airtel implements Sim {
    public void calling() {
        System.out.println("Airtel Calling...");
    }
}

class Jio implements Sim {
    public void calling() {
        System.out.println("Jio Calling...");
    }
}

class Service {
    private Sim sim;

    // Constructor Injection
    public Service(Sim sim) {
        this.sim = sim;
    }

    public void getService() {
        sim.calling();
    }
}

public class App {
    public static void main(String[] args) {
        Service service = new Service(new Airtel()); // inject Airtel
        service.getService();

        Service service2 = new Service(new Jio()); // inject Jio
        service2.getService();
    }
}
```

Benefits:

- Service doesn't care which Sim it gets (Airtel or Jio).
- We just **inject dependency**.
- **Loosely coupled** → easy to extend and test.

Example With Spring IoC Container:

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctx = new ClassPathXmlApplicationContext("Beans.xml");
        Sim_service ss = ctx.getBean(Sim_service.class);
        ss.calling();
    }
}
```

```

http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- bean definitions here -->
    <bean id="airtel" class="in.Mock.Airtel"/>
    <bean id="jio" class="in.Mock.Jio" />
    <bean id="service" class="in.Mock.Sim_service" scope="prototype">
        <constructor-arg ref="airtel"/>
    </bean>
</beans>

```

- Spring creates all objects (Airtel, Jio, Service).
- IoC container injects Airtel into Service.
- If we change to jio, no Java code change is required → just update **Beans.xml**.

Dependency Injection (DI)

Dependency Injection is the process of **injecting dependent objects into a target object** instead of creating them manually. Dependencies are provided (injected) by the **Spring IoC container**.

Why Dependency Injection?

- Promotes **Loose Coupling** → One class doesn't depend on the direct creation of another class.
- **Easy Testing** → We can swap mock objects for testing.
- **Configurable** → Just change config/annotation, not the Java code.
- **Reusability** → One class can be reused with different implementations.

Types of Dependency Injection in Spring

1. **Field Injection** → using variables (@Autowired)
2. **Setter Injection** → using setter methods
3. **Constructor Injection** → using constructors

Example Without DI (Tightly Coupled)

```

class SBI {
    public int ROI() {
        return 3;
    }
    public String location() {
        return "Bangalore";
    }
}

class BankService {
    private SBI bank = new SBI(); // tightly coupled with SBI

    public void getBankDetails() {
        System.out.println(bank.ROI());
        System.out.println(bank.location());
    }
}

public class App {
    public static void main(String[] args) {
        BankService service = new BankService();
        service.getBankDetails();
    }
}

```

Problem:

- BankService is locked to **SBI** only.
- To use **RBI**, we must edit BankService code.

Example With Constructor Injection

```
interface IBank {
    int ROI();
    String location();
}

class SBI implements IBank {
    public int ROI() { return 3; }
    public String location() { return "Bangalore"; }
}

class RBI implements IBank {
    public int ROI() { return 4; }
    public String location() { return "Mysore"; }
}

class BankService {
    private IBank bank;

    // Constructor Injection
    public BankService(IBank bank) {
        this.bank = bank;
    }

    public void getBankDetails() {
        System.out.println("ROI: " + bank.ROI());
        System.out.println("Location: " + bank.location());
    }
}

public class App {
    public static void main(String[] args) {
        BankService service1 = new BankService(new SBI());
        service1.getBankDetails();

        BankService service2 = new BankService(new RBI());
        service2.getBankDetails();
    }
}
```

Benefit:

- Just pass any IBank implementation → no code change in BankService.

Example With Setter Injection

```
interface IBank {
    int ROI();
    String location();
}

class SBI implements IBank {
    public int ROI() { return 3; }
    public String location() { return "Bangalore"; }
}

class RBI implements IBank {
    public int ROI() { return 4; }
    public String location() { return "Mysore"; }
}
```

```

class BankService {
    private IBank bank;

    // Setter Injection
    public void setBank(IBank bank) {
        this.bank = bank;
    }

    public void getBankDetails() {
        System.out.println("ROI: " + bank.ROI());
        System.out.println("Location: " + bank.location());
    }
}

public class App {
    public static void main(String[] args) {
        BankService service = new BankService();
        service.setBank(new SBI()); // injecting SBI
        service.getBankDetails();

        service.setBank(new RBI()); // injecting RBI
        service.getBankDetails();
    }
}

```

Example With Field Injection (@Autowired)

```

public interface IPayment {
    public boolean processPayment();
}

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

```

```

@Service
public class PaymentService {
    // @Autowired
    // @Qualifier("dc")
    IPayment payment;

    // @Autowired
    public PaymentService(IPayment payment) {
        // TODO Auto-generated constructor stub

        this.payment=payment;
    }

    public PaymentService() {
        // TODO Auto-generated constructor stub
    }

    @Autowired
    @Qualifier("dc")
    public void setPayment(IPayment payment) {
        this.payment=payment;
    }

    public void doPayment() {
        if(payment.processPayment()) {
            System.out.println("payment processed");
        }
    }
}

```

```

import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Repository;

@Repository("cc")
@Primary
public class CreditCard implements IPayment{

    @Override
    public boolean processPayment() {
        System.out.println("Payment processing using Credit Card");
        return true;
    }
}

import org.springframework.stereotype.Repository;

@Repository("dc")
public class DebitCard implements IPayment{

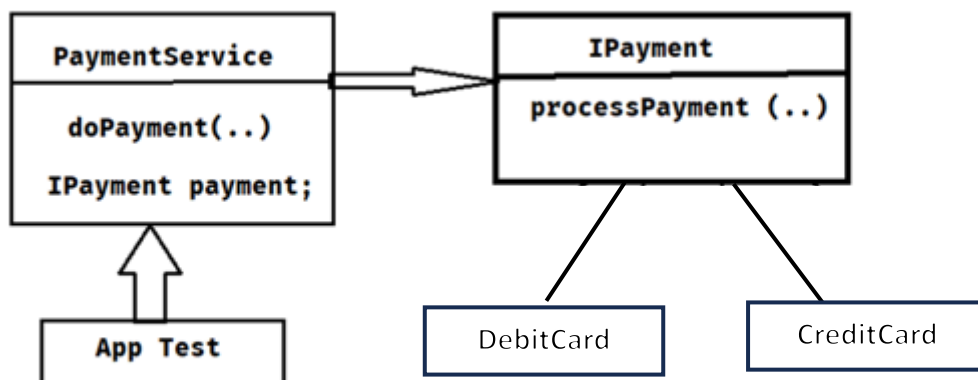
    @Override
    public boolean processPayment() {
        System.out.println("Payment processing using debit Card");
        return true;
    }
}

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import in.Mock.Config;

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctx = new AnnotationConfigApplicationContext
            (Config.class);
        PaymentService ps = ctx.getBean(PaymentService.class);
        ps.doPayment();
    }
}

```



Inversion of Control (IoC)

- IoC = Principle where the control of object creation and dependency management is inverted (given to the container, not the programmer).
- Normally in Java, we (developers) create objects using new.
- With IoC, the Spring IoC Container creates and manages objects (called Spring Beans).

Real-Life Analogy

Without IoC:

You go to a shop, buy vegetables, cook food yourself.

With IoC:

You go to a restaurant, order food → the restaurant (container) takes care of ingredients, cooking, and serving. You just consume it.

IoC in Plain Java (Without Spring)

```
class Airtel {
    public void calling() {
        System.out.println("Airtel Calling...");
    }
}

class Jio {
    public void calling() {
        System.out.println("Jio Calling...");
    }
}

class Service {
    private Airtel airtel = new Airtel(); // directly creating object

    public void getService() {
        airtel.calling();
    }
}

public class App {
    public static void main(String[] args) {
        Service service = new Service();
        service.getService();
    }
}
```

Problem:

- Service is tightly coupled with Airtel.
- If you want Jio, you must change code.

IoC with Constructor Injection

```
interface Sim {
    void calling();
}

class Airtel implements Sim {
    public void calling() {
        System.out.println("Airtel Calling...");
    }
}

class Jio implements Sim {
    public void calling() {
        System.out.println("Jio Calling...");
    }
}

class Service {
    private Sim sim;

    // Dependency is injected (IoC principle)
    public Service(Sim sim) {
        this.sim = sim;
    }

    public void getService() {
        sim.calling();
    }
}

public class App {
    public static void main(String[] args) {
        Service service1 = new Service(new Airtel()); // Inject Airtel
        service1.getService();

        Service service2 = new Service(new Jio()); // Inject Jio
        service2.getService();
    }
}
```

Control of object creation (Airtel, Jio) is outside the Service class → Inversion of Control.

IoC with Spring XML Configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans http://www.springframewc

    <!-- bean definitions here -->
    <bean id="air" class="in.awez.Airtel"></bean>
    <bean id="jio" class="in.awez.Jio"></bean>
    <bean id="car" class="in.awez.Sim_Services">
        <constructor-arg ref="air"/>

    </bean>
</beans>

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext
            ("Beans.xml");

        Sim_Services s = context.getBean(Sim_Services.class);
        s.calling();
    }
}
```


- IoC container (ApplicationContext) creates beans defined in XML.
- It injects Airtel into Service without us writing new.
- If we want Jio, just change ref="jio" in XML.

IoC with Annotations (Modern Approach)

```
public interface Sim {
    void calling();
}

import org.springframework.stereotype.Component;

@Component
class Airtel implements Sim {
    public void calling() { System.out.println
        ("Airtel Calling..."); }
}

import org.springframework.stereotype.Component;

@Component
class Jio implements Sim {
    public void calling() { System.out.println
        ("Jio Calling..."); }
}

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

@Component
class Service {
    @Autowired
    private Sim sim; // Spring injects implementation

    public void getService() {
        sim.calling();
    }
}

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan
class AppConfig {}
```

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);

        Service service = context.getBean(Service.class);
        service.getService(); // Airtel or Jio (depending on config)
    }
}
```

Singleton Classes

- A **Singleton** is a class that allows **only one object** (instance) to be created in memory during the entire program execution.
- Any request for an object of that class returns the **same instance**.

Why Singleton?

- To avoid multiple objects for the same purpose (saves memory).
- Ensures **only one shared resource** is used everywhere.
- Example:
 - Database Connection Pool
 - Logger (one instance to log everywhere)
 - Configuration Manager

How to create singleton classes in java?

- We can create singleton classes in 3 ways:

(a) Lazy Initialization

- Object created only when needed.

```
class College {
    private static College c = null;

    private College() { } // private constructor prevents object creation

    public static College getInstance() {
        if (c == null) {
            c = new College();
        }
        return c;
    }
}

public class SingletonTest1 {
    public static void main(String[] args) {
        College c1 = College.getInstance();
        College c2 = College.getInstance();
        College c3 = College.getInstance();

        System.out.println(c1);
        System.out.println(c2);
        System.out.println(c3);
    }
}
```

```
Console X
SingletonTest1 [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.exe (24 Sept 2025, 10:44:52 am - 10
College@24d46ca6
College@24d46ca6
College@24d46ca6
```

(b) Static Block Initialization

Instance created at class loading time.

```
class A1 {
    private static A1 instance;

    static {
        instance = new A1();
        System.out.println("Static block executed");
    }

    private A1() {
        System.out.println("Constructor called");
    }

    public static A1 getInstance() {
        return instance;
    }
}

public class SingletonTest2 {
    public static void main(String[] args) {
        A1 a1 = A1.getInstance();
        A1 a2 = A1.getInstance();
        System.out.println(a1 == a2); // true
    }
}
```

```
Console ×
<terminated> SingletonTest2 [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.exe (24 Sep
Constructor called
Static block executed
true
```

(c) Eager Initialization

- Instance created immediately when the class loads.

```
class A2 {
    private static final A2 instance = new A2();

    private A2() {
        System.out.println("Constructor called");
    }

    public static A2 getInstance() {
        return instance;
    }
}

public class SingletonTest3 {
    public static void main(String[] args) {
        A2 obj1 = A2.getInstance();
        A2 obj2 = A2.getInstance();
        System.out.println(obj1 == obj2); // true
    }
}
```

```
Console ×
<terminated> SingletonTest3 [Java Application] C:\Program Files\Java\jdk-23\bin\javaw.e
Constructor called
true
```

Spring Bean Scopes:

- **Bean Scope** defines **how many objects (instances)** of a Spring Bean will be created by the IoC container.
- By default, Spring beans are **singleton** (only one instance per container).
- You can change the scope depending on the requirement.

Types of Bean Scopes (Core)

1. **singleton** → Default scope. Only **one object** is created per Spring container.
2. **prototype** → A **new object** is created every time you call `getBean()`.
3. **request** → One bean per **HTTP request** (used in web apps).
4. **session** → One bean per **HTTP session** (used in web apps).

Example: Singleton Scope (default):

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans http://www.springframework.org/

<!-- bean definitions here -->
<bean id="airtel" class="in.Mock.Airtel"/>
<bean id="jio" class="in.Mock.Jio" />
<bean id="service" class="in.Mock.Sim_service" scope="singleton">
  <constructor-arg ref="airtel"/>
</bean>
</beans>
```

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctx = new ClassPathXmlApplicationContext("Beans.xml");
        Sim_service ss = ctx.getBean(Sim_service.class);
        Sim_service ss1 = ctx.getBean(Sim_service.class);
        System.out.println(ss.hashCode()); //same hashCode
        System.out.println(ss1.hashCode()); //same hashCode
    }
}
```

```
Console X
<terminated> Test (10) [Java Application] C:\Users\Asus\Downloads\sts\sts-4.31.0.RELEASE\plugin
Airtel constructor
Jio Constructor
1 para const
360062456
360062456
```

- **ApplicationContext** loads **Beans.xml**.
- **IoC container** creates only **one object** of Service.
- Both **s1** and **s2** point to the same object.
- Printing **hashCode()** shows the same value (example:360062456).

Example: Prototype Scope:

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans http://www.springframework.org/

  <!-- bean definitions here -->
  <bean id="airtel" class="in.Mock.Airtel"/>
  <bean id="jio" class="in.Mock.Jio" />
  <bean id="service" class="in.Mock.Sim_service" scope="prototype">
    <constructor-arg ref="airtel"/>
  </bean>
</beans>
```

```
package in.Mock;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctx = new ClassPathXmlApplicationContext("Beans.xml");
        Sim_service ss = ctx.getBean(Sim_service.class);
        System.out.println(ss.hashCode()); //different hashCode
        Sim_service ss1 = ctx.getBean(Sim_service.class);
        System.out.println(ss1.hashCode()); //different hashCode
    }
}
```

```
Console X
<terminated> Test (10) [Java Application] C:\Users\Asus\Downloads\sts\sts-4.31.0.RELEASE\plugins\org.eclipse

Airtel constructor
Jio Constructor
1 para const
360062456
1 para const
1790421142
```

- IoC container reads Beans.xml.
- It doesn't pre-create a singleton — instead, every time you call `getBean()`, a **new object** is created.
- s1 and s2 are different objects.
- Printing `hashCode()` shows **different values** (example: 360062456 vs 1790421142).

Spring Autowiring

- **Autowiring** is the process by which Spring **automatically identifies and injects dependencies** into a bean, instead of us manually specifying them.
- IoC container does this by looking at **variable names, types, or constructors**.

Without autowiring:

You must tell Spring which bean to inject.

With autowiring:

Spring figures it out automatically.

Autowiring Modes in Spring

1. **byName** → Matches variable name with bean id.
2. **byType** → Matches variable type with bean class.
3. **constructor** → Uses constructor arguments to inject dependency.
4. **no** → Default; no autowiring. (You must configure manually).

Example:

```
public interface Icompany {  
    public int active();  
    public void state();  
}
```

```
public class Airtel implements Icompany{  
  
    @Override  
    public int active() {  
        System.out.println("Airtel Sim Active");  
        return 1;  
    }  
  
    @Override  
    public void state() {  
        // TODO Auto-generated method stub  
        System.out.println("state=karnataka");  
    }  
}
```

```
public class Jio implements Icompany{  
  
    @Override  
    public int active() {  
        System.out.println("Jio Sim Active");  
        return 1;  
    }  
  
    @Override  
    public void state() {  
        // TODO Auto-generated method stub  
        System.out.println("state=gujarat");  
    }  
}
```

```

public class Sim {
    private Icompany com;

    public Sim() {
        // TODO Auto-generated constructor stub
    }

    public Sim(Icompany com) {
        // TODO Auto-generated constructor stub
        System.out.println("constructor called");
        this.com=com;
    }

    public void setCom(Icompany com) {
        System.out.println("setter called");
        this.com=com;
    }

    public void calling() {
        if(com.active()>0) {
            System.out.println("you can call");
        }else {
            System.out.println("sim not active");
        }
    }
}

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctxt = new ClassPathXmlApplicationContext("Beans.xml");
        Sim s = ctxt.getBean(Sim.class);
        s.calling();
    }
}

```

1: Autowiring byName

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="com" class="in.Mock.AutoWired.Airtel"/>
    <bean id="jio" class="in.Mock.AutoWired.Jio"/>
    <bean id="sim" class="in.Mock.AutoWired.Sim" autowire="byName">
    </bean>
</beans>

```

- IoC looks at Service class.
- Finds variable `private Icompany com;`
- Looks for a bean with `id = "com"` → injects that bean.

2: Autowiring byType

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="air" class="in.Mock.AutoWired.Airtel"/>
  <bean id="jio" class="in.Mock.AutoWired.Jio"/>
  <bean id="sim" class="in.Mock.AutoWired.Sim" autowire="byType">
  </bean>
</beans>
```

- IoC checks the **type** of variable Sim.
- If **exactly one implementation** (Airtel or Jio) is present, it injects automatically.
- If **multiple beans** implement the same interface → IoC gets confused (throws exception).

Solution:

- Use primary="true" in XML:

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="air" class="in.Mock.AutoWired.Airtel" primary="true"/>
  <bean id="jio" class="in.Mock.AutoWired.Jio"/>
  <bean id="sim" class="in.Mock.AutoWired.Sim" autowire="byType">
  </bean>
</beans>
```

- Disable one with autowire-candidate="false".

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="air" class="in.Mock.AutoWired.Airtel" autowire-candidate="false"/>
  <bean id="jio" class="in.Mock.AutoWired.Jio"/>
  <bean id="sim" class="in.Mock.AutoWired.Sim" autowire="byType">
  </bean>
</beans>
```


3: Autowiring by constructor:

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="air" class="in.Mock.AutoWired.Airtel"/>
    <bean id="jio" class="in.Mock.AutoWired.Jio"/>
    <bean id="sim" class="in.Mock.AutoWired.Sim" autowire="constructor">
    </bean>
</beans>
```

- IoC finds constructor in Service (Sim sim).
- Matches parameter type (Sim) with available beans.
- Injects automatically.

If multiple beans match, same ambiguity as byType → solve with primary or @Qualifier.

Annotation based approach(@Autowired):

```
public interface IFather {
    public void getPocketMoney();
}
```

```
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
@ComponentScan
public class AppConfig {

    public AppConfig() {
        super();
        System.out.println("Cons of SppConfig class");
    }

}
```

```
public class Test {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);
        FamilyService fs = context.getBean(FamilyService.class);
        fs.depositAmount();
    }
}
```

```

@Repository
public class Son implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("punith : 3k");
    }

}

@Repository
public class Daughter implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("divya : 2k");
    }

}

@Service
public class FamilyService {
    @Autowired //field injection
    private IFather f;

    @Autowired //setter injection
    public void setF(IFather f) {
        System.out.println("setter method");
        this.f = f;
    }

    @Autowired //constructor injection
    public FamilyService(IFather f) {
        super();
        System.out.println("one parameter const");
        this.f = f;
    }

    public FamilyService() {
        super();
        System.out.println("zero para const");
    }

    public void depositAmount() {
        f.getPocketMoney();
        System.out.println("random method");
    }

}

```

- Here ambiguity occurs

Resolving Ambiguity:

1. Use @Primary: Add @Primary to either Daughter or Son class as per your requirement

```
@Repository
@Primary
public class Daughter implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("divya : 2k");
    }

}
```

2. Pass the property variable name from the Target class in the @Repository(" ") of the dependent class

```
@Repository("f")
public class Son implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("punith : 3k");
    }

}
```

3. Use @Qualifier:

```
@Repository("ddao")
@Primary
public class Daughter implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("divya : 2k");
    }

}

@Repository("sdao")
public class Son implements IFather{

    @Override
    public void getPocketMoney() {
        // TODO Auto-generated method stub
        System.out.println("punith : 3k");
    }

}
```

```

@Service
public class FamilyService {
    @Autowired //field injection
    @Qualifier("ddao")
    private IFather f;

    @Autowired //setter injection
    public void setF(IFather f) {
        System.out.println("setter method");
        this.f = f;
    }

    @Autowired //constructor injection
    public FamilyService(IFather f) {
        super();
        System.out.println("one parameter const");
        this.f = f;
    }

    public FamilyService() {
        super();
        System.out.println("zero para const");
    }

    public void depositAmount() {
        f.getPocketMoney();
        System.out.println("random method");
    }
}

```

- Here if you don't use @Qualifier the class with @Primary will be considered

Annotations in Spring:

(A) Stereotype Annotations (to mark beans)

- @Component → generic Spring bean
- @Service → service layer bean (business logic)
- @Repository → DAO (Data Access Object) bean (DB operations)
- @Controller → web MVC controller
- @RestController → REST API controller (@Controller + @ResponseBody)

(B) Configuration Annotations (to configure Spring itself)

- @Configuration → marks a class as config class (replaces XML)
- @ComponentScan → tells Spring which packages to scan for beans
- @Bean → manually define a bean inside a config class

(C) Dependency Injection Annotations

- @Autowired → inject dependency automatically
- @Qualifier → resolve ambiguity between multiple beans
- @Primary → mark one bean as the default when multiple exist
- @Value → inject values from properties

Spring Bean Life Cycle

- A **Spring Bean** is any Java class managed by the **IoC container**.
- Spring gives us hooks (special methods) to execute code:
 - **When the bean is created (initialization phase)** → e.g., open DB connection.
 - **When the bean is destroyed (destruction phase)** → e.g., close DB connection.

Phases of Bean Life Cycle

1. **Object Creation** → Spring IoC container creates the bean object.
2. **Dependency Injection** → Dependencies are injected into the bean.
3. **Initialization** → Custom init method executes.
4. **Bean Ready for Use** → Application uses the bean.
5. **Destruction** → Custom destroy method executes (when container shuts down).

Bean Life Cycle Approaches

(A) Declarative Approach (XML)

(B) Programmatic Approach (Interfaces)

(C) Annotation Approach

(A) Declarative Approach (XML):

```
public class UserDao {
    public void init() {
        System.out.println("Init: Getting DB Connection...");
    }

    public void getData() {
        System.out.println("Fetching data from DB...");
    }

    public void destroy() {
        System.out.println("Destroy: Closing DB Connection...");
    }
}

public class Test {
    public static void main(String[] args) {
        ConfigurableApplicationContext context =
            new ClassPathXmlApplicationContext("Beans.xml");

        UserDao dao = context.getBean(UserDao.class);
        dao.getData();

        context.close(); // triggers destroy method
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userDao" class="in.Mock.XmlBased.UserDao"
          init-method="init" destroy-method="destroy"/>

</beans>
```

(B) Programmatic Approach (Interfaces)

Spring provides two interfaces:

- **InitializingBean** → override **afterPropertiesSet()** (init)
- **DisposableBean** → override **destroy()** (cleanup)

```
@Component
public class UserDao implements InitializingBean, DisposableBean{

    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("Init method... Getting DB connection...");
    }

    public void getData() {
        System.out.println("Fetching data from DB...");
    }

    @Override
    public void destroy() throws Exception {
        System.out.println("Destroy method... Closing DB connection...");
    }

}

public class Test {

    public static void main(String[] args) {
        AnnotationConfigApplicationContext ctx =
            new AnnotationConfigApplicationContext(AppConfig.class);
        UserDao u = ctx.getBean(UserDao.class);
        u.getData();
        ctx.close();
    }

}
```

(C) Annotation Approach:

Use:

- **@PostConstruct** → initialization
- **@PreDestroy** → cleanup

```
@Component
public class UserDao {
    @PostConstruct
    public void init() {
        System.out.println("Getting DB connection...");
    }

    public void getData() {
        System.out.println("Fetching from DB");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("Closing DB connection...");
    }

}
```